

- [EKS Upgrade Guide: 1.29 → 1.31 \(Zero-Downtime\)](#)
- [Part 1: Procedure, Pre-Checks & Execution](#)
 - [CRITICAL RULE: EKS Upgrades Are Sequential](#)
 - [UPGRADE OVERVIEW](#)
 - [PHASE 1: PRE-UPGRADE CHECKS](#)
 - [1.1 API Deprecation Audit](#)
 - [1.2 Add-on Compatibility Matrix](#)
 - [1.3 Backup Everything](#)
 - [1.4 Validate Pod Disruption Budgets \(PDBs\)](#)
 - [1.5 Pre-Upgrade Health Snapshot](#)
 - [PHASE 2: UPGRADE 1.29 → 1.30](#)
 - [Step 1: Control Plane Upgrade \(~25-40 minutes\)](#)
 - [Step 2: Upgrade EKS Add-ons](#)
 - [Step 3: Node Group Rolling Upgrade \(Zero-Downtime\)](#)
 - [Step 4: Validation After 1.30 Upgrade](#)
 - [PHASE 3: UPGRADE 1.30 → 1.31](#)

EKS Upgrade Guide: 1.29 → 1.31 (Zero-Downtime)

Part 1: Procedure, Pre-Checks & Execution

CRITICAL RULE: EKS Upgrades Are Sequential

1.29 → 1.30 → 1.31 (you CANNOT skip versions)

Each hop = Control Plane upgrade → Add-on upgrades → Node Group upgrade → Validation

UPGRADE OVERVIEW

UPGRADE SEQUENCE	
Phase 1: PRE-UPGRADE CHECKS (2-3 days before)	- API deprecation audit - Add-on compatibility check - Velero backup - PDB validation
Phase 2: UPGRADE 1.29 → 1.30	- Step 1: Control Plane upgrade (~25-40 min) - Step 2: Add-ons upgrade (CoreDNS, kube-proxy, VPC CNI, EBS CSI) - Step 3: Node Group rolling upgrade - Step 4: Validation
Phase 3: UPGRADE 1.30 → 1.31	- Repeat Steps 1-4 - Final validation
Phase 4: POST-UPGRADE VALIDATION	- Application health checks - Monitoring verification - Performance baseline comparison

PHASE 1: PRE-UPGRADE CHECKS

1.1 API Deprecation Audit

Critical deprecations by version:

Version	Deprecated API	Replacement	Impact
1.29	<code>flowcontrol.apiserver.k8s.io/v1beta2</code> removed	<code>v1beta3</code> or <code>v1</code>	FlowSchema, PriorityLevelConfig
1.30	None critical	—	Smooth upgrade
1.31	<code>flowcontrol.apiserver.k8s.io/v1beta3</code> removed	<code>v1</code>	FlowSchema, PriorityLevelConfig

How to check for deprecated APIs in your cluster:

```
# Install kubent (Kube No Trouble)
brew install kubent
# OR
sh -c "$(curl -sSL https://git.io/install-kubent)"

# Scan cluster for deprecated APIs
kubent

# Output example:
# >>> Deprecated APIs removed in 1.30 <<<
# KIND              NAMESPACE   NAME              API_VERSION
# FlowSchema        -            system-leader     flowcontrol.apiserver.k8s.io/v1beta2

# Alternative: Use kubectl to find resources using old APIs
kubectl get --raw /metrics | grep apiserver_requested_deprecated_apis
```

Fix deprecated resources BEFORE upgrading:

```
# Export, update apiVersion, re-apply
kubectl get flowschema system-leader -o yaml > flowschema.yaml
# Edit: change apiVersion from v1beta2 to v1
kubectl apply -f flowschema.yaml
```

1.2 Add-on Compatibility Matrix

Check current add-on versions:

```
# List all EKS add-ons and their versions
aws eks describe-addon --cluster-name my-cluster --addon-name coredns \
  --query 'addon.addonVersion' --output text

aws eks describe-addon --cluster-name my-cluster --addon-name kube-proxy \
  --query 'addon.addonVersion' --output text

aws eks describe-addon --cluster-name my-cluster --addon-name vpc-cni \
  --query 'addon.addonVersion' --output text

aws eks describe-addon --cluster-name my-cluster --addon-name aws-ebs-csi-driver \
  --query 'addon.addonVersion' --output text

# List compatible versions for target K8s version
aws eks describe-addon-versions --kubernetes-version 1.30 \
  --addon-name coredns --query 'addons[].addonVersions[].addonVersion'
```

Recommended add-on versions:

Add-on	EKS 1.29	EKS 1.30	EKS 1.31
CoreDNS	v1.11.1-eksbuild.6	v1.11.1-eksbuild.8	v1.11.3-eksbuild.1
kube-proxy	v1.29.x-eksbuild.x	v1.30.x-eksbuild.x	v1.31.x-eksbuild.x
VPC CNI	v1.16.x	v1.18.x	v1.19.x
EBS CSI	v1.28.x	v1.30.x	v1.33.x

Note: Always check latest compatible versions via `aws eks describe-addon-versions` as these change frequently.

1.3 Backup Everything

```
# Velero full cluster backup
velero backup create pre-upgrade-1.29-backup \
  --include-namespaces '*' \
  --snapshot-volumes=true \
  --ttl 720h \
  --wait

# Verify backup
velero backup describe pre-upgrade-1.29-backup
velero backup logs pre-upgrade-1.29-backup

# Export critical resources manually (belt and suspenders)
kubectl get deployments --all-namespaces -o yaml > deployments-backup.yaml
kubectl get services --all-namespaces -o yaml > services-backup.yaml
kubectl get configmaps --all-namespaces -o yaml > configmaps-backup.yaml
kubectl get ingress --all-namespaces -o yaml > ingress-backup.yaml
kubectl get crds -o yaml > crds-backup.yaml

# Backup Helm releases
helm list -A > helm-releases.txt
for release in $(helm list -A -q); do
  helm get values $release -n $(helm list -A | grep $release | awk '{print $2}')
> helm-values-${release}.yaml
done

# Backup ArgoCD applications
kubectl get applications -n argocd -o yaml > argocd-apps-backup.yaml
```

1.4 Validate Pod Disruption Budgets (PDBs)

PDBs are critical for zero-downtime upgrades. Without them, K8s can drain all pods of a service simultaneously.

```

# List all PDBs
kubectl get pdb --all-namespaces

# Verify each critical service has a PDB
# GOOD PDB example:
kubectl get pdb -n payments
# NAME             MIN AVAILABLE   MAX UNAVAILABLE   ALLOWED DISRUPTIONS
# payment-pdb      N/A             1                 1

# If missing, create PDBs for critical services:
cat <<EOF | kubectl apply -f -
apiVersion: policy/v1
kind: PodDisruptionBudget
metadata:
  name: payment-pdb
  namespace: payments
spec:
  maxUnavailable: 1           # At most 1 pod can be down during drain
  selector:
    matchLabels:
      app: payment-service
EOF

# Verify replicas - PDB needs replicas > maxUnavailable
kubectl get deployment payment-service -n payments
# Must have replicas >= 2 if maxUnavailable = 1

```

1.5 Pre-Upgrade Health Snapshot

```

#!/bin/bash
# pre_upgrade_health.sh - Run before upgrade, save output for comparison

echo "=== CLUSTER INFO ==="
kubectl cluster-info
kubectl version --short

echo "=== NODE STATUS ==="
kubectl get nodes -o wide

echo "=== ALL PODS STATUS ==="
kubectl get pods --all-namespaces | grep -v Running | grep -v Completed

echo "=== DEPLOYMENTS NOT READY ==="
kubectl get deployments --all-namespaces | awk '$3 != $4'

echo "=== PDB STATUS ==="
kubectl get pdb --all-namespaces

echo "=== RESOURCE USAGE ==="
kubectl top nodes
kubectl top pods --all-namespaces --sort-by=memory | head -20

echo "=== DAEMONSETS ==="
kubectl get daemonsets --all-namespaces

```

```
echo "=== SERVICES (LoadBalancer) ==="
kubectl get svc --all-namespaces | grep LoadBalancer

echo "=== CERTIFICATES ==="
kubectl get certificates --all-namespaces 2>/dev/null

echo "=== STORAGE ==="
kubectl get pv,pvc --all-namespaces
```

PHASE 2: UPGRADE 1.29 → 1.30

Step 1: Control Plane Upgrade (~25-40 minutes)

What happens during control plane upgrade:

- AWS replaces API server instances one at a time behind the EKS-managed NLB
- API server may be briefly unavailable (seconds, not minutes)
- **Existing workloads continue running** — pods are NOT affected
- kubectl may return intermittent errors during the switch

Using AWS CLI:

```
# Start control plane upgrade
aws eks update-cluster-version \
  --name my-cluster \
  --kubernetes-version 1.30

# Monitor upgrade status
watch -n 30 'aws eks describe-cluster --name my-cluster \
  --query "cluster.\
{status:status,version:version,platformVersion:platformVersion}" \
  --output table'

# Wait for completion
aws eks wait cluster-active --name my-cluster
echo "Control plane upgrade to 1.30 complete!"
```

Using Terraform:

```
resource "aws_eks_cluster" "main" {
  name      = "my-cluster"
  version   = "1.30" # Changed from "1.29"
  role_arn  = aws_iam_role.cluster.arn

  vpc_config {
```

```
    subnet_ids          = var.private_subnet_ids
    endpoint_private_access = true
    endpoint_public_access = false
  }
}
```

```
terraform plan    # Review changes – should show only version update
terraform apply   # Takes 25–40 minutes
```

Step 2: Upgrade EKS Add-ons

Order matters: CoreDNS → kube-proxy → VPC CNI → EBS CSI

```
# 1. CoreDNS
aws eks update-addon \
  --cluster-name my-cluster \
  --addon-name coredns \
  --addon-version v1.11.1-eksbuild.8 \
  --resolve-conflicts OVERWRITE

# Wait for CoreDNS
aws eks wait addon-active --cluster-name my-cluster --addon-name coredns

# 2. kube-proxy
aws eks update-addon \
  --cluster-name my-cluster \
  --addon-name kube-proxy \
  --addon-version v1.30.0-eksbuild.3 \
  --resolve-conflicts OVERWRITE

# 3. VPC CNI
aws eks update-addon \
  --cluster-name my-cluster \
  --addon-name vpc-cni \
  --addon-version v1.18.1-eksbuild.1 \
  --resolve-conflicts OVERWRITE

# 4. EBS CSI Driver
aws eks update-addon \
  --cluster-name my-cluster \
  --addon-name aws-ebs-csi-driver \
  --addon-version v1.30.0-eksbuild.1 \
  --resolve-conflicts OVERWRITE

# Verify all add-ons
aws eks list-addons --cluster-name my-cluster --output table
for addon in coredns kube-proxy vpc-cni aws-ebs-csi-driver; do
  echo "---- $addon ----"
  aws eks describe-addon --cluster-name my-cluster --addon-name $addon \
    --query 'addon.{version:addonVersion,status:status}' --output table
done
```

Step 3: Node Group Rolling Upgrade (Zero-Downtime)

This is where pods actually move. PDBs ensure zero service disruption.

Strategy: Blue-Green Node Group (safest):

```
# Option A: Managed Node Group – in-place update
aws eks update-nodegroup-version \
  --cluster-name my-cluster \
  --nodegroup-name my-node-group \
  --kubernetes-version 1.30

# This performs a rolling update:
# 1. Launches new node with 1.30 AMI
# 2. Cordons old node (no new pods scheduled)
# 3. Drains old node (respecting PDBs)
# 4. Pods reschedule on new node
# 5. Terminates old node
# 6. Repeats for each node

# Monitor:
watch -n 10 'kubectl get nodes -o wide'
```

Option B: Blue-Green Node Group (zero-risk):

```
# 1. Create NEW node group with 1.30
aws eks create-nodegroup \
  --cluster-name my-cluster \
  --nodegroup-name my-node-group-v130 \
  --kubernetes-version 1.30 \
  --node-role-arn $NODE_ROLE_ARN \
  --subnets $SUBNET_IDS \
  --instance-types t3.xlarge \
  --scaling-config minSize=3,maxSize=10,desiredSize=3

# 2. Wait for new nodes to be Ready
kubectl get nodes --watch

# 3. Cordon old nodes (stop scheduling new pods)
for node in $(kubectl get nodes -l eks.amazonaws.com/nodegroup=my-node-group -o name); do
  kubectl cordon $node
done

# 4. Drain old nodes one by one (respects PDBs)
for node in $(kubectl get nodes -l eks.amazonaws.com/nodegroup=my-node-group -o name); do
  kubectl drain $node --ignore-daemonsets --delete-emptydir-data --timeout=300s
  echo "Drained $node - waiting 60s for pods to stabilize..."
  sleep 60
done

# Verify all deployments are healthy
```

```
kubectl get deployments --all-namespaces | awk '$3 != $4'
```

done

```
# 5. Verify all pods running on new nodes
kubectl get pods --all-namespaces -o wide | grep -v "my-node-group-v130"
# Should return empty - all pods on new nodes

# 6. Delete old node group
aws eks delete-nodegroup \
  --cluster-name my-cluster \
  --nodegroup-name my-node-group
```

Step 4: Validation After 1.30 Upgrade

```
# Cluster version
kubectl version --short

# All nodes on 1.30
kubectl get nodes -o wide
# v1.30.x should show for all nodes

# All pods healthy
kubectl get pods --all-namespaces | grep -v Running | grep -v Completed

# DNS working
kubectl run dns-test --image=busybox --rm -it -- nslookup kubernetes.default
kubectl run dns-test --image=busybox --rm -it -- nslookup payment-
service.payments.svc.cluster.local

# CoreDNS healthy
kubectl get pods -n kube-system -l k8s-app=kube-dns

# Networking
kubectl run net-test --image=busybox --rm -it -- wget -q0- http://payment-
service.payments.svc.cluster.local/health

# Storage (if using EBS)
kubectl get pv,pvc --all-namespaces
# All should be Bound

# Application health endpoints
curl -s https://api.company.com/health | jq .
```

PHASE 3: UPGRADE 1.30 → 1.31

Repeat the exact same process:

1. Control plane: `--kubernetes-version 1.31`
2. Add-ons: Update to 1.31-compatible versions

3. Node groups: Rolling update or blue-green

4. Validation

1.31-specific changes to watch:

Change	Impact	Action
<code>flowcontrol.apiserver.k8s.io/v1beta3</code> removed	FlowSchema resources	Update to v1 before upgrade
Sidecar containers GA	Native sidecar support	Review init container configurations
AppArmor GA	Security profiles	Test if AppArmor profiles work correctly
Persistent Volume Last Phase Transition	PV status field changes	Update monitoring if tracking PV status

Continue to Part 2 for: Common Errors, Troubleshooting, and Rollback Mechanisms